

701 Personal project Report

Tonny Chen

Declaration: Artificial intelligence tools, including ChatGPT, were used to assist with HDL debugging, testbench generation, code commenting, and grammar correction. The final design decisions, implementation, verification, and technical interpretation were reviewed and validated by the author

1. Theoretical Background

Digital signal processing (DSP) lies at the intersection of mathematics, electrical engineering, and computer engineering, and focuses on the processing of discrete-time signals. It is widely used in embedded systems, particularly in applications that require real-time processing of sampled data. From an algorithmic perspective, a DSP operation can be interpreted as a mathematical function, similar to a function implemented in a high-level programming language such as Python. However, when implemented in hardware, the same operation behaves as a dedicated signal-processing circuit, where digital samples pass through a datapath to produce results with predictable latency.

To satisfy throughput and latency requirements, Application-Specific Processors (ASPs) are often used instead of relying solely on a central processing unit (CPU). Common DSP functions include filtering, transform-based analysis, correlation, and spectral analysis. Within a heterogeneous multiprocessor system-on-chip (HMPSoC), suitable DSP functions for ASP implementation include moving-average filtering, FIR filtering, autocorrelation, cross-correlation, peak detection, zero-crossing detection, threshold detection, RMS calculation, energy estimation, convolution, matched filtering, and FFT-based spectral analysis.

These functions are well suited to ASP encapsulation because they operate on continuous streams of sampled data and typically involve repetitive arithmetic or comparison operations that can be efficiently mapped onto dedicated hardware datapaths. By implementing these functions as ASPs, the HMPSoC can offload computationally intensive DSP tasks from the CPU, resulting in higher throughput, lower latency, and more predictable real-time behaviour.

2. Frequency Analyser System Overview

Following team discussions, Team 6 decided to adopt a hardware architecture based on the COMPSYS 723 low-cost frequency relay project. This architecture was selected because it aligns well with the existing FreeRTOS-based software framework and is already familiar to all team members. The signal-processing pipeline is partitioned into multiple Application-Specific Processors (ASPs), with each ASP responsible for a specific DSP function. Within this division of work, Tonny is responsible for the correlation calculator, Eric is responsible for the peak detector, and Solomon is responsible for the signal generator and moving average filter.

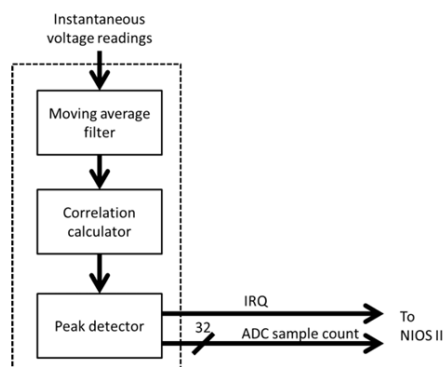


Figure 1. Flow chart of the signal processing algorithm for reference point detection.

Figure 1: frequency analyser system diagram

The frequency analyser measures the real-time frequency of incoming signals and forwards the detected waveform period to the Nios II softcore processor. The Nios II processor then calculates the corresponding signal frequency and performs the required software-level control actions. From a system-level perspective, the design consists of a signal generator front end, a moving-average filter, a correlation calculator, a peak detector, and the Nios II softcore processor. The signal generator emulates real-time sampled data that would normally be provided by an analogue-to-digital converter (ADC). The moving-average filter is then used to smooth the sampled signal, while the correlation calculator further suppresses noise and higher-order harmonics. An offset is also introduced to reduce the effect of the negative trough region of the waveform, allowing the peak detector to identify the waveform period more reliably. The peak detector outputs the period as the number of ADC samples between successive detected peaks. Since the ADC sampling interval is 1/16 Hz, this sample count can be forwarded to the Nios II softcore processor, which converts it into the corresponding signal frequency.

3. Correlation Algorithm

Zooming into the correlation algorithm, from a mathematics perspective, correlation is a type of statistic relationship between two random variables. The intuition can be understood through its similarity to the dot product of two discrete sequences. By summing the element-wise products of the two sequences, the correlation value provides a measure of how closely the sequences are related. A large positive correlation indicates that the two sequences are strongly aligned and vary in a similar manner. A value close to zero suggested that there is little meaningful relationship between them, as positive and negative contributions tend to cancel out. Conversely, a large negative correlation indicates a strong inverse relationship, where one sequence tends to increase when the other decrease.

Autocorrelation measures the similarity between a signal and a time-shifted version of itself. In the proposed correlation calculator core, autocorrelation is used to identify the repeating pattern of the incoming waveform and produce a processed waveform with one dominant reference point per cycle. This provides a more reliable input for the peak detection stage, since direct peak detection is vulnerable to random noise, waveform distortion, DC offset, and higher-order harmonic components.

By correlating the signal with a delayed version of itself, the correlation calculator reduces the influence of random noise and emphasises the periodic structure of the waveform. The resulting reference-point-based waveform allows the peak detector to operate on a more stable signal representation, improving the reliability of the estimated signal period.

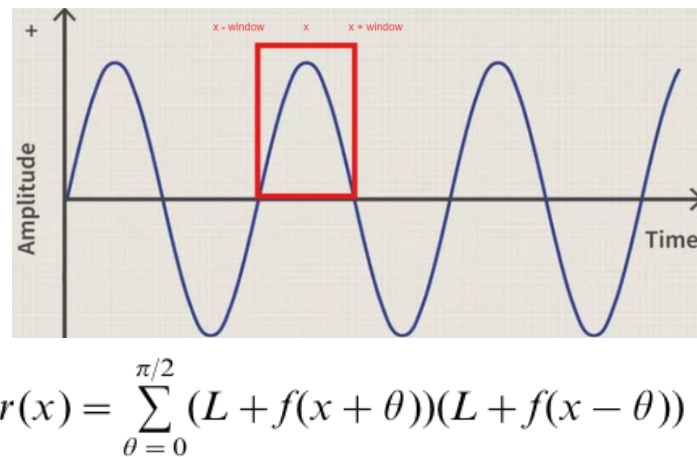


Figure 2(a): correlation window on a sinusoidal waveform and Figure 2(b): correlation equation, where the L is the offset, 2 x theta is the width of the window [1].

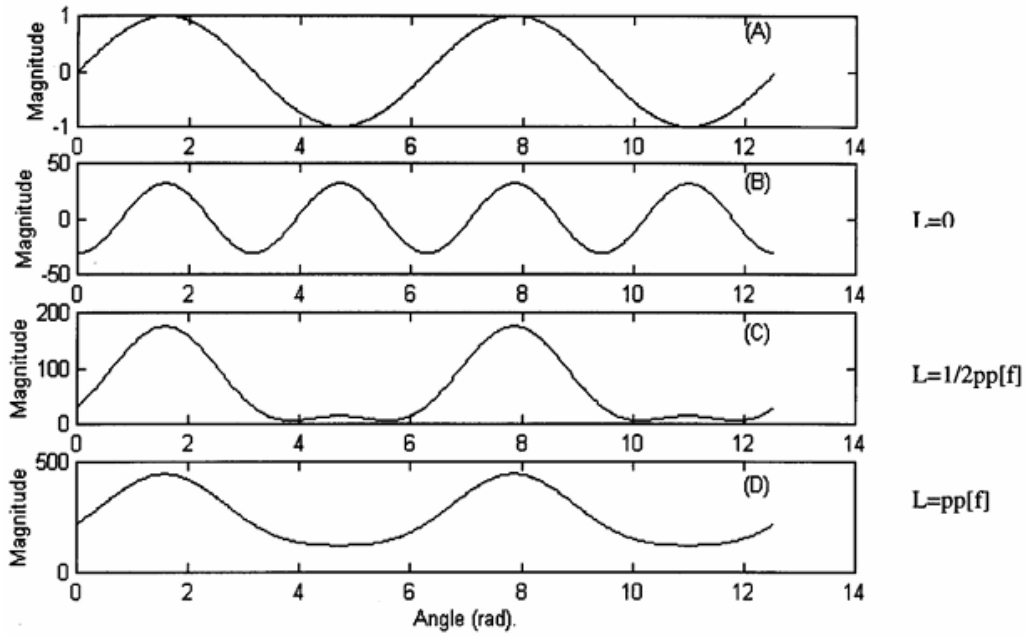


Figure 3 illustrates the output waveform with different offset L value[1]

The correlation calculator evaluates the similarity between samples located symmetrically around a reference position x . For each offset θ within the selected window, the sample to the right of x , $f(x + \theta)$, is multiplied by the corresponding sample to the left of x , $f(x - \theta)$. Before multiplication, a constant offset L is added to both samples to reduce the contribution of the waveform trough. The products are then accumulated to generate the correlation value $r(x)$ [1]. In figure 2 (a), When the reference position x is aligned with the crest region of the waveform, the samples on both sides of x are highly similar, resulting in a large correlation value. This produces one dominant reference point per cycle, which can then be used by the peak detector to estimate the signal period more reliably.

$$\text{Correlation} = (f(x) + L) \times (f(x-1) + L) + (f(x+1) + L) \times (f(x-2) + L) + (f(x+2) + L) \times (f(x-3) + L) \dots (f(x+n-1) + L) \times (f(x-n) + L)$$

Figure 4 illustrates the DSP autocorrelation equation

Moving from the continuous-time algorithm to a DSP implementation, the function behaves in a similar way. For each chosen reference position, the read-data pointers scan through the selected memory window and access sample pairs located symmetrically around the reference point. An offset value L is also added to them. These paired samples are then multiplied and accumulated using MAC operations to generate the correlation value for that reference position.

The correlation calculator core mainly consists of two parts: the address generator unit and the multiply-accumulate (MAC) unit. The address generator stores the key control registers, including the correlation window size, the current reference memory address, and a counter used to generate the paired sample addresses. For each counter value, the address generator produces two memory addresses corresponding to the two samples used in the correlation pair. In the implemented equation, the paired samples are selected as $f(x+k)$ and

$f(x-k-1)$, where k ranges from 0 to $n-1$. These addresses are sent to the averaged data RAM, and the returned samples are then passed to the MAC unit.

The MAC unit contains several key registers, including `offset_h`, `current_ave_data`, `multiplicand_temp`, `correlation_pair_product`, and `correlation_temp`. For each sample pair, the first returned sample is added with the configured offset h and stored in `current_ave_data`. In the following clock cycle, the second returned sample is also added with h , while the previous value is transferred into `multiplicand_temp`. This forms the two paired values $(f(x+k)+h)$ and $(f(x-k-1)+h)$. The two values are then multiplied, and the product is accumulated into `correlation_temp`. After all sample pairs within the selected correlation window have been processed, `correlation_temp` represents the correlation value for the current reference position.

4.3. Correlation Control Path: A Moore Machine implementation

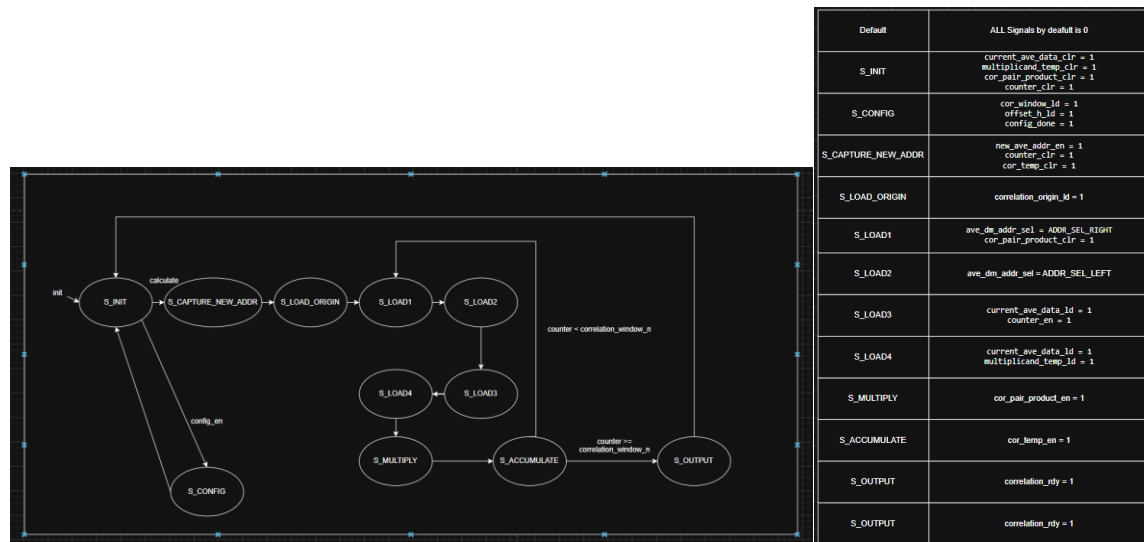


Figure 7(a) illustrates the Moore's state diagram and Figure 7(b) illustrates the control signal generated by each state

In total, the Moore machine implementation contains 11 states. The operation starts in the `S_INIT` state. When the `config_en` signal is asserted, the machine enters the `S_CONFIG` state, where the correlation window size register and the offset L register can be configured. When a `calculate` signal is received, the machine enters the `S_CAPTURE_NEW_ADDR` state, where the current reference memory address is captured and the correlation output register is cleared. The next state is `S_LOAD_ORIGIN`, which updates the reference address used later to compute the left and right memory addresses.

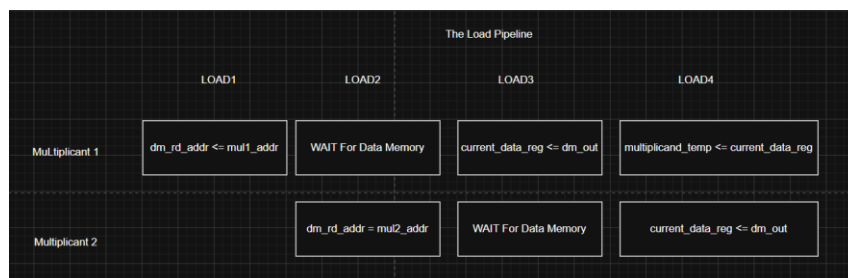


Figure 7 illustrates the LOAD states pipeline

The machine then enters four load states, which are used to load the two multiplicand values through a simple pipeline. Since the Quartus RAM IP requires one clock cycle to return the requested data, the pipeline uses this delay to write the next memory address into the data memory while storing the returned sample into the appropriate register. The loaded samples are stored in the `current_ave_data` register and the `multiplicand_temp`

register. This allows the two paired values required by the correlation equation to be formed across multiple clock cycles.

After both multiplicands have been loaded, the machine enters the S_MULTIPLY state, where the two values are multiplied together. It then enters the S_ACCUMULATE state, where the product is added to the correlation accumulator. The counter is then compared with the configured correlation window size. If the counter has not reached the window size, the machine continues processing the next sample pair. Once the counter is equal to or greater than the window size, the machine proceeds to the S_OUTPUT state and outputs the final correlation value.

4.4. Correlation ASP Capabilities and Instructions

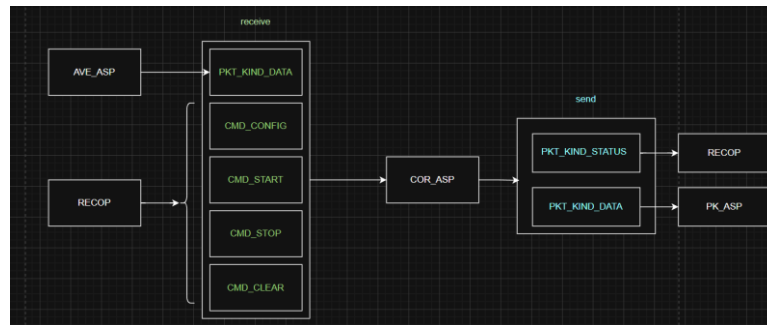


Figure 8 illustrates the supported receive and send command

The Correlation ASP supports two operational modes: direct passthrough mode and correlation mode. During a system reset, which may be initiated by the CMD_CLEAR command from ReCOP, the Correlation ASP clears its control flags, resets the relevant internal control registers, and returns to direct passthrough mode. In direct passthrough mode, packets received from the Average ASP are forwarded directly to the Peak Detection ASP without being processed by the correlation calculator.

In correlation mode, the correlation result is 36 bits wide, while the packet payload field supports only 20 bits. Therefore, each correlation result is transmitted using two packets. The lower 20 bits are sent first, followed by a second packet containing the upper 16 bits. This packet-splitting mechanism allows the full correlation result to be transferred through the existing packet format without modifying the payload width.

The CMD_START command switches the Correlation ASP from direct passthrough mode to correlation mode. In this mode, incoming averaged data is processed by the correlation calculator before being forwarded to the Peak Detection ASP. The CMD_STOP command returns the ASP to direct passthrough mode. The CMD_CONFIG command allows ReCOP to update key correlation parameters, including the offset value, offset_h, and the correlation window size. After each control or configuration command is received from ReCOP, the Correlation ASP generates a status packet and sends it back to ReCOP as an acknowledgement.

packet type	address	data			
		kind[1:0]	command[27:24]	sender_address[23:16]	payload[19:0]
data packet from average asp to correlation asp	COR_ASP_ADDR	PKT_KIND_DATA			unused 4 bits payload
lower 20 bits data packet from correlation asp to peak asp	PK_ADDR	PKT_KIND_DATA			payload
upper 16 bits data packet from correlation asp to peak asp	PK_ADDR	PKT_KIND_DATA			unused 4 bits payload
parameter configuration packet from recop to correlation asp	COR_ASP_ADDR	PKT_KIND_CMD	CMD_CONFIG		
command start packet from recop to correlation asp	COR_ASP_ADDR	PKT_KIND_CMD	CMD_START		
command stop packet from recop to correlation asp	COR_ASP_ADDR	PKT_KIND_CMD	CMD_STOP		
command clear packet from recop to correlation asp	COR_ASP_ADDR	PKT_KIND_CMD	CMD_CLEAR		
status packet from correlation asp to recop	RECOP_ADDR	PKT_KIND_STATUS			unused 4 bits payload

Figure 9 illustrates the packet formatting for supported command

4.5. NOC Interface: the decoder and the encoder

The decoder and encoder form the packet-level communication interface of the Correlation ASP. The decoder receives incoming packets, extracts the packet type, command code, tag, and payload fields, and generates the corresponding internal control signals. It also manages the ASP operating mode and determines whether incoming data packets are forwarded directly to the next ASP or written into data memory for correlation processing. The encoder formats all outgoing packets, including direct passthrough data packets, correlation result packets, and status acknowledgement packets.

To support acknowledgement packets returned to ReCOP, a handshake mechanism is implemented between the decoder, the correlation calculator Moore machine, and the encoder. For example, when a CMD_CONFIG packet is received, the decoder extracts the configuration payload and updates either the correlation window register or the offset register. The decoder then asserts config_en through an internal pending register, causing the correlation calculator Moore machine to enter the S_CONFIG state and load the updated configuration values.

Once the configuration operation is complete, the Moore machine asserts a one-clock-cycle `config_done` pulse back to the decoder. Since this completion signal is only valid for a single clock cycle, the decoder captures it into a persistent `config_done_status_reg`. This prevents the completion event from being lost before the acknowledgement packet is transmitted. At the same time, the decoder asserts `status_pending_reg`, indicating that a status response must be sent back to ReCOP.

The encoder monitors the `status_pending` signal. When this signal is asserted, the encoder generates a status packet containing the relevant status flags, such as `config_done_reg`, `direct_passthrough_en`, and `reset_done_reg`, and transmits it to ReCOP. After the packet has been generated, the encoder asserts `status_transmit_done` for one clock cycle. This signal is fed back to the decoder, allowing the decoder to clear the pending status and completion registers. This handshake mechanism converts short completion pulses from the Moore machine into stable acknowledgement packets, ensuring reliable communication between the Correlation ASP and ReCOP.

5. Testbench and Verification

The design was validated at two levels. First, the correlation calculator core and data memory were tested together as the top-level entity. Second, the integrated design was verified with the NoC interface included. Both validation stages were checked using manual waveform analysis and assertion-based verification. Where AI-assisted assertion code was used, it was reviewed and tested by the author before being included in the verification process. The design and testbench were iterated multiple times to ensure functional correctness.

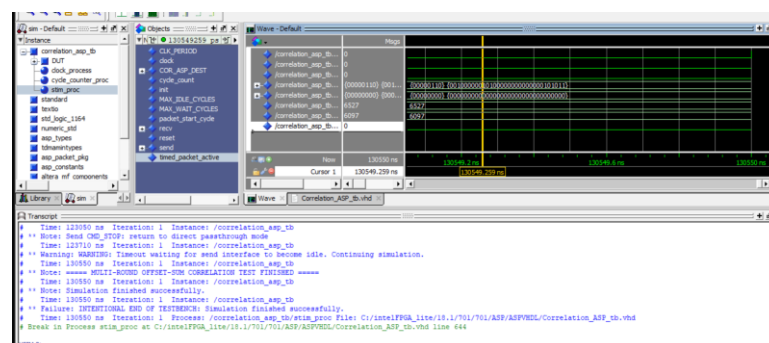


Figure 10 illustrates the log message from testbench

The ModelSim testbench verifies the COR-ASP at both packet-interface and datapath levels. The test sequence confirms that configuration packets are decoded correctly, configuration registers are updated, input samples are written into memory, the address counter and correlation counter advance as expected, and the multiplier-

accumulator datapath produces the expected correlation result. The testbench also verifies that result packets and status acknowledgement packets are generated only after the corresponding operation has completed. This ensures that the ASP does not produce false output packets before calculation completion and that reset or clear commands return the design to a known direct passthrough state.

6. Performance and Resource Usage

The Correlation ASP was synthesised in Quartus to evaluate its hardware resource utilisation and maximum operating frequency. The design was targeted to the Cyclone V 5CSEMA5F31C6 FPGA device. The current ReCOP system operates at 25 MHz, while the intended future integration target is 50 MHz. According to the Quartus Timing Analyzer, the Correlation ASP achieves a maximum operating frequency, F_{max} , of 213.77 MHz. This is significantly higher than both the current 25 MHz system clock and the planned 50 MHz target clock, indicating that the design has sufficient timing margin for real-time operation.

The high F_{max} is mainly achieved through the multi-cycle structure of the correlation calculator. Instead of performing address generation, RAM access, multiplication, accumulation, and control updates within a single large combinational path, the design separates these operations across multiple Moore machine states. The controller contains 11 states in total, covering configuration, address capture, RAM loading, multiplication, accumulation, comparison, and output generation. This reduces the amount of combinational logic between registers in each clock cycle, allowing the circuit to achieve a much higher maximum clock frequency. Therefore, the design trades additional latency for improved timing performance.

The complete Correlation ASP uses 215 combinational LUTs, 273 dedicated logic registers, 8192 block memory bits, 1 DSP block, and 83 pins. Overall, the resource usage is relatively low for the target FPGA. The averaged data memory accounts for all 8192 block memory bits, while the MAC operation unit uses the single DSP block required for multiplication. The correlation calculator is the largest logic component, using 144 combinational LUTs and 168 registers. Within the calculator, the address generator contributes 79 LUTs and 36 registers, while the MAC operation unit contributes 57 LUTs, 120 registers, and 1 DSP block. The decoder and encoder are smaller packet-interface modules, using 42 LUTs and 58 registers, and 29 LUTs and 47 registers, respectively.

The latency of the correlation operation depends on the configured correlation window size. In this implementation, the lower result packet is generated after $6 \times \text{window} + 6$ clock cycles. This latency accounts for the repeated load, multiply, accumulate, and loop-control operations required for each sample pair within the correlation window.

Since the correlation output is 36 bits wide and the packet payload is limited to 20 bits, the encoder transmits the result using two packets. The lower 20 bits are sent first, followed by the upper 16 bits one clock cycle later. Therefore, the upper result packet is generated one cycle after the lower result packet, giving a latency of $6 \times \text{window} + 7$ clock cycles.

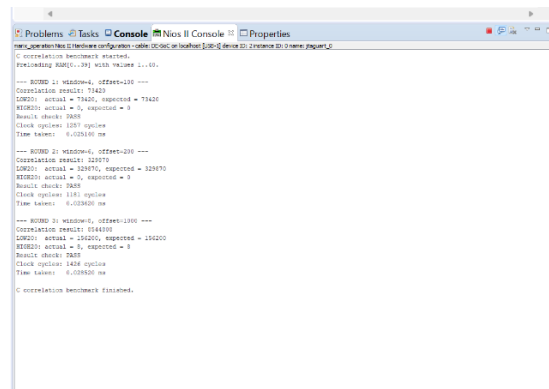
For example, when the correlation window size is set to 8, the lower result packet is generated after $6 \times 8 + 6 = 54$ clock cycles. The upper result packet is then generated one clock cycle later, after 55 clock cycles. This matches the simulation result, where the upper correlation packet is observed after 55 clock cycles. Therefore, including the transmission of both result packets, the Correlation ASP produces one complete 36-bit correlation result every $6 \times \text{window} + 7$ clock cycles.

The throughput is not the limiting factor in this application, because the ASP is part of a time-based frequency analyser where the input rate is determined by the sampling period rather than by the maximum processing speed of the hardware.

Overall, the Correlation ASP meets the required timing performance with a large margin. The achieved F_{max} of 213.77 MHz is substantially higher than the planned 50 MHz ReCOP clock, meaning that the design is not timing-limited under the target system frequency. Although the multi-cycle Moore machine increases the number of cycles required to produce each correlation result, it keeps each state simple and enables high

operating frequency with low hardware resource usage. Compared with direct peak detection, the correlation-based approach requires additional memory, address generation logic, and a MAC datapath. However, this cost is acceptable because the correlation stage provides a more stable reference waveform for the Peak Detection ASP and improves the robustness of period detection under noisy or distorted input conditions.

8. Software Comparison on Nio II



The measured Nios II software execution times were 25.140 μ s for window = 4, 23.620 μ s for window = 6, and 28.520 μ s for window = 8. These measurements represent the time required for the processor to complete the correlation calculation in software.

The Quartus Timing Analyzer reports an Fmax of 213.77 MHz for the Correlation ASP. This value should not be interpreted as the normal operating frequency of the full ReCOP system. Instead, it indicates that the ASP has a large timing margin above both the current 25 MHz system clock and the future 50 MHz target clock. If the Correlation ASP were operated at its theoretical maximum frequency, the window = 8 hardware latency would be approximately 0.257 μ s. However, the more realistic system-level comparison should be made using the current and planned ReCOP clock frequencies.

9. Conclusion

This project successfully implemented and verified a Correlation Application-Specific Processor (COR-ASP) for a hardware-based frequency analyser system. The COR-ASP was designed to improve the robustness of period detection by applying an autocorrelation-based preprocessing stage before peak detection. By comparing sample pairs around a reference point and accumulating their offset-adjusted products, the design generates one dominant reference point per waveform cycle. This provides a more stable input for the Peak Detection ASP, particularly when the incoming waveform contains noise, distortion, DC offset, or higher-order harmonic components.

The implemented COR-ASP consists of a packet decoder, averaged data memory, correlation calculator core, and packet encoder. The correlation calculator was implemented using a multi-cycle Moore machine, which separates address generation, RAM access, multiplication, accumulation, and output generation across multiple states. This design choice increases processing latency in terms of clock cycles but significantly improves timing performance by reducing the combinational logic depth in each cycle. The ASP also supports direct passthrough mode, correlation mode, configurable window size and offset values, status acknowledgement packets, and two-packet transmission for the 36-bit correlation result.

The design was validated at both datapath and system-interface levels. The correlation calculator core was first tested together with the data memory, and the complete ASP was then verified with the NoC packet interface included. The ModelSim testbench confirmed correct configuration decoding, sample reception, memory addressing, multiplier and accumulator behaviour, result packet generation, status acknowledgement generation, and reset behaviour. The verification also confirmed that no false output packet is generated before calculation completion.

Synthesis results from Quartus show that the design uses relatively low hardware resources on the Cyclone V FPGA, including 215 combinational LUTs, 273 registers, 8192 block memory bits, and one DSP block. The achieved $F_{\max}$ of 213.77 MHz is well above both the current 25 MHz ReCOP clock and the planned 50 MHz target clock, demonstrating that the COR-ASP has sufficient timing margin for real-time operation. For a correlation window size of 8, the hardware requires 55 clock cycles to produce the complete 36-bit result. This corresponds to 2.20 μs at 25 MHz and 1.10 μs at 50 MHz, compared with 28.520 μs for the equivalent Nios II software implementation.

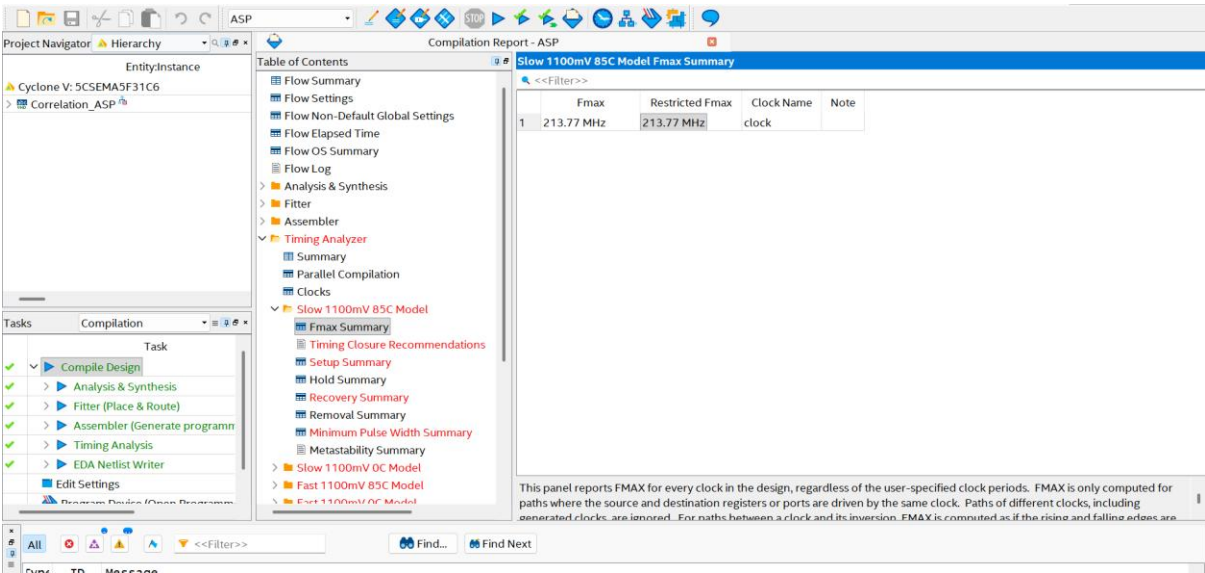
Overall, the Correlation ASP meets the functional, timing, and integration requirements of the frequency analyser system. Compared with a pure software implementation, the dedicated hardware design provides lower latency, avoids occupying the Nios II processor during repeated correlation calculations, and offers more predictable real-time behaviour. Although the correlation-based approach requires additional memory, address generation logic, and a MAC datapath compared with direct peak detection, this cost is justified by the improved stability of the waveform reference point and the increased robustness of period measurement.

Reference List:

[1] Z. Salcic and R. Mikhael, “A new method for instantaneous power system frequency measurement using reference points detection,” Electric Power Systems Research, vol. 55, no. 2, pp. 97–102, 2000.

Appendix:

fmax



Resource Usage

Analysis & Synthesis Resource Utilization by Entity							
<<Filter>>							
	Compilation Hierarchy Node	Combinational ALUTs	Dedicated Logic Registers	Block Memory Bits	DSP Blocks	Pins	Virtual Pins
1	▼ Correlation_ASP	215 (0)	273 (0)	8192	1	83	0
1	▼ AverageFilteredData:u_memory	0 (0)	0 (0)	8192	0	0	0
1	▼ altsyncram:altsyncram_component	0 (0)	0 (0)	8192	0	0	0
1	altsyncram_p5u3:auto_generated	0 (0)	0 (0)	8192	0	0	0
2	CorAspDecoder:u_decoder	42 (42)	58 (58)	0	0	0	0
3	CorAspEncoder:u_encoder	29 (29)	47 (47)	0	0	0	0
4	▼ Correlation_Calculator:u_correlation_calculator	144 (0)	168 (0)	0	1	0	0
1	address_generator:u_address_generator	79 (79)	36 (36)	0	0	0	0
2	mac:u_operation	57 (57)	120 (120)	0	1	0	0
3	multicycle_moore_machi...lticycle_moore_machine	8 (8)	12 (12)	0	0	0	0

Note: For table entries with two numbers listed, the numbers in parentheses indicate the number of resources of the given type used by the specific entity alone. They computed for